

Johdatus tekoälyyn

Sami Sieranoja

Päivitetty viimeksi: 2026-05-18 19:46:08+03:00

Lisensoitu Creative Commons CC0 1.0 Universal Public Domain Dedication -lisenssillä:
<https://creativecommons.org/publicdomain/zero/1.0/>

Sisällys

Tämä dokumentti on tuotettu osittain tekoälyn avulla. Katso seuraava loki vuorovaikutuksesta OpenAI Codexin kanssa: <https://github.com/SamiSieranoja/aint/blob/main/text/codex-log.txt>

1	Tekoäly	2
1.1	Tekoälyn määritelmä	2
1.2	Tekoälyn määrittämisen vaikeudet	2
1.3	Kapea ja yleinen tekoäly	2
1.4	Itsensä parantaminen ja singulariteetti	3
2	Neuroverkot	4
2.1	Lineaarinen regressio	4
2.2	Yksittäinen neuroni	4
2.3	Esimerkkejä yksittäisestä neuronista	5
2.4	Pistetulot	6
2.5	Matriisi-vektorikertolasku	6
2.6	Kerrosten laskenta matriisioperaatioina	7
2.7	Esimerkki: kaksikerroksinen verkko	7
2.8	Tensorit	8
2.9	Yhteenvedo	9
3	Gradient Descent	10

Chapter 1

Tekoäly

1.1 Tekoälyn määritelmä

Tekoäly (AI) tutkii ja rakentaa laskennallisia järjestelmiä, jotka suorittavat tehtäviä, joita ihmisillä pidetään älykkyyteen liittyvinä. Tällaisia tehtäviä ovat esimerkiksi havainnointi, kielen käyttö, päättely, suunnittelu, oppiminen, päätöksenteko ja toiminta monimutkaisissa ympäristöissä.

Tämä näkemys seuraa John McCarthyn ja hänen kanssakirjoittajiensa alkuperäistä vuoden 1955 Dartmouth-ehdotuksen muotoilua. Siinä tekoälyn ongelma kuvattiin koneen saamisena käyttäytymään tavoilla, joita kutsuttaisiin älykkäiksi, jos ihminen käyttäytyisi samoin.

1.2 Tekoälyn määrittelymisen vaikeudet

Tekoälyn määrittely on vaikeaa, koska älykkyys ei ole yksittäinen ja täsmällisesti mitattava ominaisuus. Ihmisen älykkyyteen kuuluu monia kykyjä, eivätkä nämä kyvyt vastaa suoraan laskennallista vaikeutta.

Yksi tärkeä havainto on, että *ihmisille helpot tehtävät voivat olla tietokoneille vaikeita*, kun taas *ihmisille vaikeat tehtävät voivat olla tietokoneille helppoja*. Esimerkiksi suurten lukujen kertolaskut ovat useimmille ihmisille hitaita ja vaikeita, mutta tietokoneille suoraviivaisia. Sen sijaan kasvojen tunnistaminen, monitulkintaisen puheen ymmärtäminen tai liikkuminen jäsentymättömässä ympäristössä on ihmisille arkipäiväistä mutta koneille teknisesti vaativaa. Tätä vastakohtaa kutsutaan usein Moravecin paradoksiksi.¹

Toinen ongelma on, että tekoälyn raja muuttuu ajan myötä. Kun tehtävä ratkaistaan luotettavasti, sitä ei usein enää pidetä tekoälynä vaan tavallisena ohjelmistotekniikkana. Optinen merkkien tunnistus, reitinsuunnittelu, roskapostin suodatus ja puheentunnistus kuvaavat tätä muutosta.

On myös niin, ettei onnistumiselle ole yhtä ainoaa kriteeriä. Jotkin määritelmät korostavat ihmismäistä käyttäytymistä, kuten Turingin testissä. Toiset korostavat rationaalista toimintaa: järjestelmän tulisi valita tavoitteidensa, tietojensa ja rajoitteidensa kannalta sopivia toimia. Nämä näkökulmat johtavat erilaisiin tutkimuskysymyksiin ja arviointimenetelmiin.

1.3 Kapea ja yleinen tekoäly

Kapea tekoäly (narrow AI), jota kutsutaan myös heikoksi tekoälyksi (weak AI), tarkoittaa järjestelmiä, jotka on suunniteltu tiettyihin tehtäviin tai tehtävuokkiin. Esimerkkejä ovat shakkiohjelma,

¹https://en.wikipedia.org/wiki/Moravec%27s_paradox

lääketieteellisten kuvien luokittelija, suosittelujärjestelmä ja puheentunnistin. Tällaiset järjestelmät voivat ylittää ihmisen suorituskyvyn omalla alueellaan, mutta niiden osaaminen rajoittuu tähän alueeseen.

Yleinen tekoäly (artificial general intelligence, AGI) tarkoittaa hypoteettista järjestelmää, jolla olisi laaja ja joustava älyllinen kyvykkyys monilla alueilla. Yleinen tekoäly ei ainoastaan suorittaisi yhtä ennalta määriteltyä tehtävää, vaan se voisi oppia uusia tehtäviä, siirtää tietoa alueelta toiselle, päätellä uusissa tilanteissa ja sopeutua muuttuviin ympäristöihin ihmisentasoisella tai sitä laajemmalla yleisyydellä.

Erottelu on tärkeä, koska kapeat järjestelmät voivat olla hyvin kyvykkäitä ilman yleistä älykkyyttä. Järjestelmä voi kääntää tekstiä, luokitella kuvia tai ratkaista matemaattisia ongelmia, mutta silti siltä voi puuttua vankka arkijärki, pitkäaikainen autonomia tai luotettava siirtyminen tuntemattomiin tilanteisiin.

Nykyisiä tekoälyjärjestelmiä pidetään edelleen kapeana tekoälynä. Yleinen kehityssuunta on kuitenkin se, että tekoälyjärjestelmistä tulee yhä yleiskäyttöisempiä.²

1.4 Itsensä parantaminen ja singulariteetti

Itsensä parantaminen (self-improvement) tekoälyssä tarkoittaa tekoälyjärjestelmän kykyä parantaa omaa suorituskyykyään. Rajoitetussa mielessä tätä tapahtuu jo koneoppimisessa: järjestelmät päivittävät parametrejaan datasta, säätävät toimintapolitiikkojaan (policies) vahvistusoppimisessa (reinforcement learning) tai käyttävät haku-avaruuden (search space) läpikäyntiä ratkaisujen parantamiseen. Tavallisesti tämä tapahtuu kuitenkin ihmiskehittäjien asettamien rajojen sisällä, kuten kiinteän arkkitehtuurin, koulutusmenettelyn, tavoitteen ja datalähteen puitteissa.

Rekursiivinen itsensä parantaminen (recursive self-improvement) on vahvempi ajatus. Se kuvaa järjestelmää, joka voisi muokata omia algoritmejaan, arkkitehtuuriaan, koulutusprosessiaan tai laitteiston käyttöään tavoilla, jotka lisäävät sen kykyä tehdä uusia parannuksia. Jos jokainen parannus tekisi seuraavasta parannuksesta helpomman, kyvykkyyden kasvu voisi periaatteessa kiihtyä.

Tavallisessa teknologisessa kehityksessä ihmiset pysyvät mukana prosessissa: he asettavat tavoitteet, suunnittelevat järjestelmät, testaavat ne ja päättävät, mitä otetaan käyttöön. Teknologinen *singulariteetti* (singularity) koskee kehittyneempää tilannetta, jossa tekoälyjärjestelmät voisivat ottaa osan näistä rooleista ja kiihdyttää omaa kehitystään. Jos prosessi nopeutuisi riittävästi, ihmisten voisi olla vaikea ymmärtää, ennustaa tai hallita sitä.

Varhainen muotoilu tästä ajatuksesta esiintyy I.J. Goodin esseessä *Speculations Concerning the First Ultraintelligent Machine* (1965). Good väitti, että kone, joka voisi ylittää ihmiset kaikissa älyllisissä toiminnoissa, voisi myös suunnitella parempia koneita. Tämä voisi tuottaa “älykkyysräjähdysen” (intelligence explosion). Siksi hän kuvasi ensimmäistä ultraälykästä konetta mahdollisesti viimeiseksi keksinnöksi, joka ihmisten tarvitsisi tehdä, jos kone pysyisi hallittavana.

²<https://www.noemamag.com/artificial-general-intelligence-is-already-here/>

Chapter 2

Neuroverkot

Tässä luvussa käsitellään:

- Neuroverkkoja (neural networks) aloittamalla yksinkertaisemmasta mallista, lineaarisesta regressiosta (linear regression), sekä sitä, miten keinotekoinen neuroni (artificial neuron) voidaan muodostaa lisäämällä lineaariseen malliin aktivaatiofunktio (activation function).
- Miten yksittäiset neuronit yhdistetään kerroksiksi (layers) ja miten kerrokset muodostavat neuroverkkoja.
- Matriisi-vektori kertolaskua (matrix-vector multiplication) ja sitä, miten se tarjoaa tiiviin tavan esittää neuroverkon laskentaa.

2.1 Lineaarinen regressio

Lineaarinen regressio (linear regression) on valvotun oppimisen perusmenetelmä numeerisen tuloksen ennustamiseen syötepiirteistä. Kun syötevektori on $x = (x_1, \dots, x_n)$, lineaarinen regressiomalli laskee

$$\hat{y} = w_1x_1 + \dots + w_nx_n + b,$$

missä $w_1, \dots, w_n$ ovat painoja (weights) ja b on bias-termi (bias term). Malli on lineaarinen syötepiirteiden suhteen: syötteen muuttaminen muuttaa ennustetta kiinteällä määrällä, jonka kyseinen paino määrää.

Koulutuksen aikana painot ja bias valitaan niin, että ennustevirhe esimerkeissä pienenee. Yleinen tavoite on keskineliövirhe (mean squared error), joka kuvaa ennustetun arvon $\hat{y}$ ja tavoitearvon y neliöllistä erotusta.

2.2 Yksittäinen neuroni

Yksittäinen keinotekoinen neuroni aloittaa samalla painotetulla summalla, jota käytetään lineaarisessa regressiossa:

$$z = w_1x_1 + \dots + w_nx_n + b.$$

Erona on, että neuroni soveltaa tämän jälkeen aktivaatiofunktioita f :

$$a = f(z).$$

Tulosta a kutsutaan neuronin aktivaatioksi (activation).

Ilman aktivaatiofunktia neuronin on vain lineaarinen malli.

Aktivaatiofunktioita on useita, mutta ReLU-funktio mainitaan ensin, koska se on yksi yksinkertaisimmista ja hyvin yleisesti käytetyistä. Sillä on kolme tavallista määritelmää:

$$\text{ReLU}(z) = \max(0, z) = \frac{z + |z|}{2} = \begin{cases} 0, & z < 0, \\ z, & z \geq 0 \end{cases}$$

ReLU palauttaa negatiivisille syönteille nollan ja positiivisille syönteille syönteiden itsensä. Se on yksinkertainen, laskennallisesti halpa ja laajasti käytetty neuroverkkojen piilokerroksissa (hidden layers).

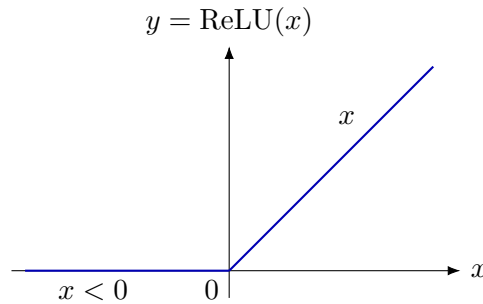


Figure 2.1: ReLU kuvaa negatiiviset syönteet nolaksi ja jättää positiiviset syönteet muuttumattomiksi.

2.3 Esimerkkejä yksittäisestä neuronista

Tarkastellaan neuronin, jolla on kolme syötettä, painot w_1, w_2, w_3 , bias b ja ReLU-aktivaatio $\sigma(z) = \text{ReLU}(z)$. Sen tulos on

$$y = \sigma(w_1x_1 + w_2x_2 + w_3x_3 + b).$$

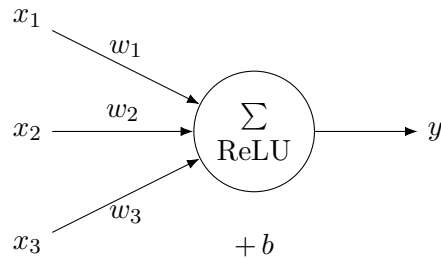


Figure 2.2: Yksittäinen neuronin laskee painotetun summan, lisää biasin ja soveltaa ReLU-funktiota.

Olkoon $w_1 = 4$, $w_2 = 1$, $w_3 = -2$ ja $b = -4$. Kun $x_1 = 3$, $x_2 = 2$ ja $x_3 = 1$,

$$y = \sigma(4 \cdot 3 + 1 \cdot 2 + (-2) \cdot 1 - 4) = \sigma(8) = 8.$$

Kun $x_1 = 1$, $x_2 = 6$ ja $x_3 = 5$,

$$y = \sigma(4 \cdot 1 + 1 \cdot 6 + (-2) \cdot 5 - 4) = \sigma(-4) = 0.$$

Molemmat esimerkit käyttävät samoja neuronin parametreja. Vain syöte muuttuu, joten painotettu summa muuttuu positiivisesta arvosta negatiiviseksi arvoksi; ReLU päästää positiivisen arvon läpi ja leikkaa negatiivisen arvon nolaksi.

2.4 Pistetulot

Lineaarisen regression ja neuronin painotettu summa voidaan kirjoittaa pistetulona (dot product). Kahdelle vektorille

$$w = (w_1, \dots, w_n), \quad x = (x_1, \dots, x_n),$$

pistetulo on

$$w \cdot x = w_1 x_1 + \dots + w_n x_n.$$

Siksi neuronin esiaktivaatioarvo voidaan kirjoittaa tiiviisti muodossa

$$z = w \cdot x + b.$$

Pistetulo mittaa syötepiirteiden painotettua yhdistelmää.

Esimerkiksi jos $a = (1, 2, 3)$ ja $b = (4, 5, 6)$, niin

$$a \cdot b = 1 \cdot 4 + 2 \cdot 5 + 3 \cdot 6 = 4 + 10 + 18 = 32.$$

2.5 Matriisi-vektorikertolasku

Usean neuronin kerros voidaan esittää painomatriisilla (weight matrix). Oletetaan, että kerroksessa on m neuronia ja jokainen neuroni saa saman syötevektorin x , jossa on n piirrettä. Painot voidaan järjestää matriisiksi

$$W = \begin{bmatrix} w_{11} & w_{12} & \cdots & w_{1n} \\ w_{21} & w_{22} & \cdots & w_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ w_{m1} & w_{m2} & \cdots & w_{mn} \end{bmatrix}.$$

Tulo Wx on vektori, jossa on yksi arvo jokaista neuronia kohti:

$$Wx = \begin{bmatrix} w_1 \cdot x \\ w_2 \cdot x \\ \vdots \\ w_m \cdot x \end{bmatrix}.$$

Matriisi-vektorikertolasku on siis toistettua pistetuloa: matriisi jaetaan riveihin, ja syötevektorin ja jokaisen rivin pistetulo lasketaan erikseen.

Esimerkiksi olkoon

$$A = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix}, \quad x = \begin{bmatrix} 1 \\ 2 \\ -1 \end{bmatrix}.$$

Tällöin

$$Ax = \begin{bmatrix} 1 \cdot 1 + 2 \cdot 2 + 3 \cdot (-1) \\ 4 \cdot 1 + 5 \cdot 2 + 6 \cdot (-1) \end{bmatrix} = \begin{bmatrix} 2 \\ 8 \end{bmatrix}.$$

2.6 Kerrosten laskenta matriisioperaatioina

Matriisien avulla kokonaisen kerroksen laskenta voidaan kirjoittaa muodossa

$$z = Wx + b,$$

missä b on nyt bias-termeistä koostuva vektori. Aktivaatiofunktiota sovelletaan sen jälkeen komponentti kerrallaan:

$$a = f(z).$$

ReLU:n tapauksessa tämä tarkoittaa arvon $\max(0, z_i)$ soveltamista jokaiseen komponenttiin z_i .

Neuroverkko koostuu useista tällaisista kerroksista. Jos $a^{(0)} = x$ on syöte, tyypillinen kerros laskee

$$z^{(\ell)} = W^{(\ell)}a^{(\ell-1)} + b^{(\ell)}, \quad a^{(\ell)} = f(z^{(\ell)}).$$

Merkintä näyttää, miksi matriisi-vektorikertolasku on keskeinen neuroverkoissa: se laskee kaikki kerroksen painotetut summat kerralla. Moderni laitteisto pystyy suorittamaan näitä matriisioperaatioita tehokkaasti, mikä on yksi syy siihen, että neuroverkot skaalautuvat suuriin malleihin ja suuriin datasetteihin.

2.7 Esimerkki: kaksikerroksinen verkko

Tarkastellaan verkkoa, jonka syöte on

$$x = \begin{bmatrix} 2 \\ 3 \end{bmatrix}, \quad W_1 = \begin{bmatrix} 1 & 4 \\ 0 & -3 \end{bmatrix}, \quad W_2 = \begin{bmatrix} -2 & 7 \\ 3 & -9 \end{bmatrix}.$$

Käytetään bias-vektoreita

$$b^{(1)} = \begin{bmatrix} -2 \\ 1 \end{bmatrix}, \quad b^{(2)} = \begin{bmatrix} 3 \\ 4 \end{bmatrix},$$

ja sovelletaan ReLU:ta jokaisen kerroksen jälkeen.

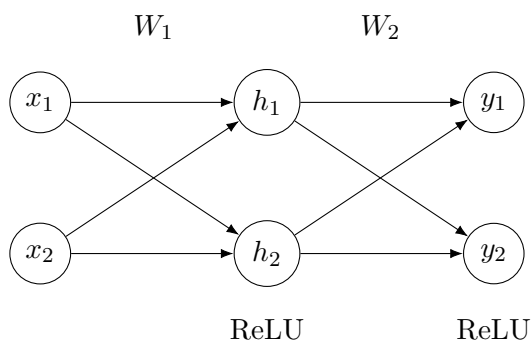


Figure 2.3: Kaksikerroksinen täysin kytketty neuroverkko, jossa on kaksi syöteyksikköä, kaksi piiloyksikköä ja kaksi tuloyksikköä.

Ensimmäinen kerros laskee

$$z^{(1)} = W_1 x + b^{(1)} = \begin{bmatrix} 1 & 4 \\ 0 & -3 \end{bmatrix} \begin{bmatrix} 2 \\ 3 \end{bmatrix} + \begin{bmatrix} -2 \\ 1 \end{bmatrix} = \begin{bmatrix} 1 \cdot 2 + 4 \cdot 3 - 2 \\ 0 \cdot 2 + (-3) \cdot 3 + 1 \end{bmatrix} = \begin{bmatrix} 12 \\ -8 \end{bmatrix}.$$

ReLU:n jälkeen

$$a^{(1)} = \text{ReLU}(z^{(1)}) = \begin{bmatrix} 12 \\ 0 \end{bmatrix}.$$

Toinen kerros laskee

$$\begin{aligned} z^{(2)} &= W_2 a^{(1)} + b^{(2)} \\ &= \begin{bmatrix} -2 & 7 \\ 3 & -9 \end{bmatrix} \begin{bmatrix} 12 \\ 0 \end{bmatrix} + \begin{bmatrix} 3 \\ 4 \end{bmatrix} \\ &= \begin{bmatrix} (-2) \cdot 12 + 7 \cdot 0 + 3 \\ 3 \cdot 12 + (-9) \cdot 0 + 4 \end{bmatrix} \\ &= \begin{bmatrix} -21 \\ 40 \end{bmatrix}. \end{aligned}$$

Kun ReLU:ta sovelletaan uudelleen, verkon tulos on

$$y = \text{ReLU}(z^{(2)}) = \begin{bmatrix} 0 \\ 40 \end{bmatrix}.$$

2.8 Tensorit

Koneoppimisessa tensori (tensor) on tietorakenne numeeriselle datalle. Sitä käytetään monissa koneoppimisen kirjastoissa, kuten PyTorchissa ja TensorFlow:ssa. Skalaarit (scalars), vektorit (vectors), matriisit (matrices) ja korkeamman ulottuvuuden taulukot voidaan kaikki nähdä tensoreina. Ulottuvuuksien määrää kutsutaan usein tensorin rangiksi (rank) tai asteeksi (order). PyTorchissa tensorit esitetään `torch.Tensor`-olioina.

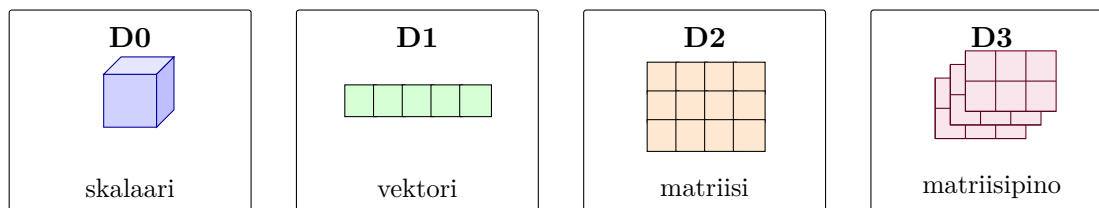


Figure 2.4: Erilaisia N -ulotteisia tensoreita arvoilla $N = 0, \dots, 3$. Arvolle N ei ole periaatteellista ylärajaa.

0D-tensori on skalaaritensori (scalar tensor). Alla oleva PyTorch-koodi luo tensorin, joka sisältää vain arvon 3.14 eikä sillä ole akseleita. Siksi sen muoto on tyhjä muoto `[]`, ei `[1]`.

```
import torch
```

```
x0 = torch.tensor(3.14)
print(x0.shape) # torch.Size([])
```

1D-tensori on vektori, kuten syötepiirteiden lista.

```
x1 = torch.tensor([5.0, 12.0, 15.0])
print(x1.shape) # torch.Size([3])
```


2D-tensori on matriisi. Esimerkiksi painomatriisi voi kuvata useita syötepiirteitä useisiin neuroneihin.

```
x2 = torch.tensor([
    [1.0, 2.0, 3.0],
    [4.0, 5.0, 6.0],
])
print(x2.shape)  # torch.Size([2, 3])
```

3D-tensori voi esittää matriisien batchin. Batch tarkoittaa joukkoa esimerkkejä, jotka käsitellään yhdessä laskennassa. Esimerkiksi kolme harmaasävykuvaa, joiden koko on 4×4 , voidaan tallentaa muotoon `[3, 4, 4]`. Tässä tapauksessa batchin koko on 3, ja jokainen batchin esimerkki on 4×4 -matriisi.

```
x3 = torch.zeros(3, 4, 4)
print(x3.shape)  # torch.Size([3, 4, 4])
```

4D-tensori on yleinen kuvankäsittelyssä, jossa data järjestetään usein muotoon

`[batch, channels, height, width]`.

Esimerkiksi kahdeksan RGB-kuvan batch, kun kuvien koko on 32×32 , on muodoltaan `[8, 3, 32, 32]`.

```
x4 = torch.zeros(8, 3, 32, 32)
print(x4.shape)  # torch.Size([8, 3, 32, 32])
```

Tensorit ovat keskeisiä neuroverkoissa, koska syötteet, tulokset, painot, aktivaatiot, gradientit (gradients) ja koulutusesimerkkien batchit esitetään kaikki tensoreina. Neuroverkkokirjastot, kuten PyTorch, on suunniteltu suorittamaan tensorioperaatioita tehokkaasti ja laskemaan gradientteja näiden operaatioiden läpi.

Modernit GPU:t käsittelevät tensoreita tallentamalla ne taulukoina muistiin ja soveltamalla monia pieniä aritmeettisia operaatioita rinnakkain. Käytännössä korkean tason tensorioperaatiot pilkotaan yleensä matriisikertolaskuksi, konvoluutioksi (convolution) ja elementtikohtaisiksi kerneleiksi (elementwise kernels). NVIDIA:n Volta-arkkitehtuurista vuonna 2017 alkaen monissa GPU:issa on ollut myös erillisiä Tensor Cores -yksiköitä, jotka on suunniteltu kiihdyttämään neuroverkoissa käytettäviä pieniä matriisin multiply-accumulate-operaatioita.¹ Vaikka ohjelmoijat siis työskentelevät eriasteisten tensorien kanssa, laitteisto suorittaa usein paloitetuja matriisioperaatioita ja niihin liittyviä rinnakkaisia kerneleitä.

2.9 Yhteenveto

Lineaarinen regressio ennustaa numeerisen arvon syötepiirteiden painotetun summan avulla. Neuronin yleistää tämän ajatuksen soveltamalla painotettuun summaan aktivaatiofunktioita. ReLU on yksinkertainen epälineaarinen aktivaatio, joka palauttaa negatiivisille syötteille nollan ja positiivisille syötteille syötteen itsensä. Pistetulot ilmaisevat painotetut summat tiiviisti, ja matriisi-vektorikertolasku laskee monta pistetuloa kerralla. Siksi neuroverkon kerrokset esitetään luontevasti matriisi-vektorilaskuina, joita seuraavat epälineaariset aktivaatiofunktioita.

¹<https://developer.nvidia.com/blog/programming-tensor-cores-cuda-9/>

Chapter 3

Gradient Descent